

PDC Assignment 4 – Building Resilient Distributed Systems

Name: [Khadijah Farooqi]
Student ID: [Bscs23128]]>

Part 1: Analyze the Mess

Problem 1 – Lost Update (Synchronization)

The root cause is the absence of any **concurrency control** in the database write path. The naive architecture follows a blind Read-Modify-Write cycle:

1. User A reads document at version N.
2. User B reads the same document at version N.
3. User A writes their changes → document is now at version N+1.
4. User B writes their changes on top of stale data → **silently overwrites User A's work**, reverting the document to a modified version of N.

This is a textbook **Lost Update anomaly**. Because neither the FastAPI route nor the database query carries a version check, the second write always wins unconditionally, making data loss invisible to the losing user.

Problem 2 – Dropped Webhook (Coordination)

Clerk delivers subscription-cancellation events as HTTP POST webhooks. The naive handler is **at-most-once**: if the network drops the packet,

Clerk retries a finite number of times; if all retries land during an outage, the event is permanently lost. The backend's `is_premium` flag stays `true` indefinitely. The two systems (Clerk's subscription state and the app's database) are now **permanently inconsistent** with no self-healing mechanism, because there is no reconciliation loop, no idempotent retry store, and no dead-letter queue.

Problem 3 – Synchronous LLM Call (Fault Tolerance)

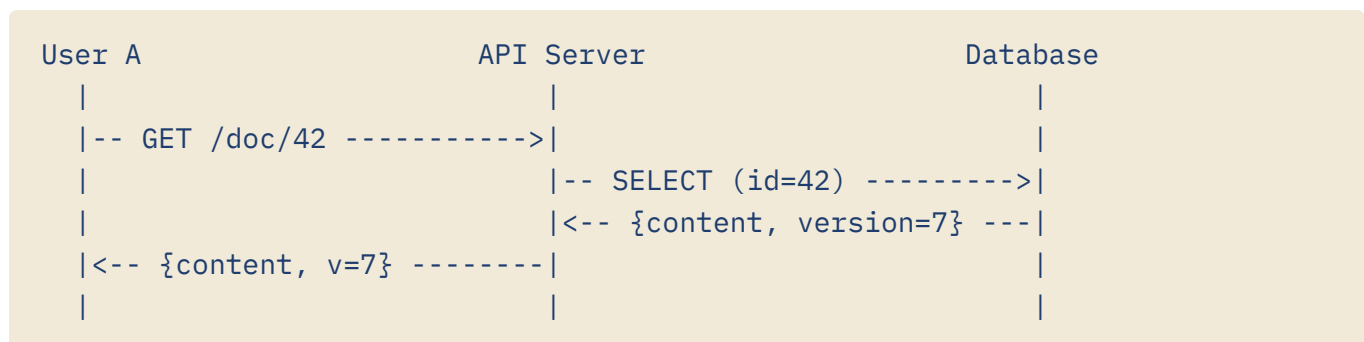
The FastAPI route calls the external LLM with a plain `await httpx.get(...)` with no timeout or fallback. When the LLM degrades, every in-flight request parks an **async task** on the event loop waiting up to 60 seconds for a response that never comes. Because Python's asyncio event loop is single-threaded, a large number of simultaneously stalled coroutines starves healthy requests of CPU scheduling time, effectively **hanging the entire server** – a single slow dependency becomes a full system outage.

Part 2: Design a Better System

Fix 1 – Optimistic Locking (Sync)

Add an integer `version` column to the documents table. Every write becomes a **conditional update**: `UPDATE documents SET content=:c, version=version+1 WHERE id=:id AND version=:expected_version`. If zero rows are affected, the client's version is stale and the server returns `HTTP 409 Conflict` instead of silently overwriting.

UML Sequence Diagram – Concurrent Edit with Optimistic Locking



```

|   [User B also fetches – also gets version=7]   |
|                                                    |
| -- PUT /doc/42 (v=7) ----->|
|                                                    |
|                                                    |-- UPDATE WHERE version=7 -->|
|                                                    |<-- 1 row affected -----|
|<-- 200 OK (v=8) -----|
|                                                    |
|   [User B submits with stale version=7]           |
|                                                    |
|   User B                                         |
| -- PUT /doc/42 (v=7) ----->|
|                                                    |-- UPDATE WHERE version=7 -->|
|                                                    |<-- 0 rows affected -----|
|<-- 409 Conflict -----|
|   (re-fetch & merge)                           |

```

The client on a 409 re-fetches the current document, performs a **three-way merge** (base=their last fetch, theirs=their edits, ours=current server state), and resubmits.

Fix 2 – Fault-Tolerant Webhook Handler (Coordination)

The handler must be **idempotent and durable**:

1. **Idempotency Key:** Clerk sends a unique `svix-id` header with every webhook. On arrival, store this ID in a `processed_webhooks` table. Before processing, check if the key already exists – if so, return `200` immediately (safe replay). This prevents duplicate processing during Clerk's automatic retries.
2. **Transactional Write:** The `is_premium = false` DB update and the idempotency key insert must happen in the **same database transaction**. Either both commit or neither does – no partial state.
3. **Dead-Letter Queue:** If the handler itself crashes after acknowledging receipt, the message is lost. A durable queue (e.g. Redis Streams or SQS) absorbs the incoming webhook, the handler consumes from the queue, and the queue retains the message until the handler explicitly acks it. Failed processing lands in a **Dead-Letter Queue (DLQ)** for manual review and replay.

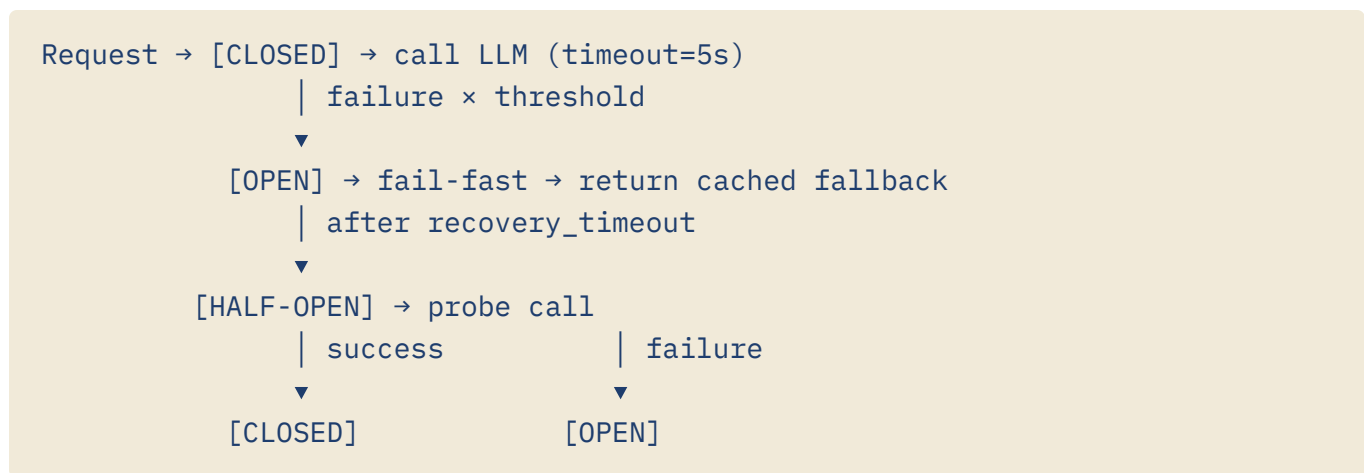
4. **Reconciliation Cron:** A scheduled job (e.g., every hour) calls the Clerk API directly to compare `is_premium` flags against Clerk's source-of-truth subscription list, patching any drift that slipped through.

Fix 3 – Circuit Breaker + Fallback (Fault Tolerance)

A **Circuit Breaker** wraps every LLM call and operates as a three-state state machine:

State	Behaviour
CLOSED	Requests flow normally. Consecutive failures increment a counter.
OPEN	Counter reached threshold. All requests fail immediately (no waiting). A cached/degraded fallback is returned.
HALF-OPEN	After a recovery timeout, one probe request is let through. Success → CLOSED; failure → OPEN.

The critical difference: in OPEN state the server returns a fallback response in **<1 ms** instead of blocking for 60 seconds. The event loop stays free for healthy requests.



Chosen for Part 3 implementation – see `app/main.py`.

CAP Theorem Trade-offs

Problem	Choice	Reasoning
Sync (Optimistic Locking)	CP – Consistency + Partition Tolerance	A 409 rejection is a brief availability loss for the losing writer, but it prevents data corruption. For a document editor, silently losing work is worse than a momentary conflict error.
Coordination (Webhook Queue)	AP – Availability + Partition Tolerance	The webhook queue buffers events during network partitions so the handler stays available. Consistency is eventual : the <code>is_premium</code> flag may lag Clerk by seconds to minutes during an outage, which is acceptable for a billing event.
Fault Tolerance (Circuit Breaker)	AP – Availability + Partition Tolerance	The circuit breaker explicitly trades consistency (serving a stale/cached fallback instead of a fresh LLM response) for availability (the server keeps responding to all users). A degraded answer is better than a hung server.

Key insight: There is no single right answer on the CAP triangle. Each sub-system has a different tolerance for staleness vs. downtime, and the architecture should reflect those domain-specific priorities rather than applying one policy globally.